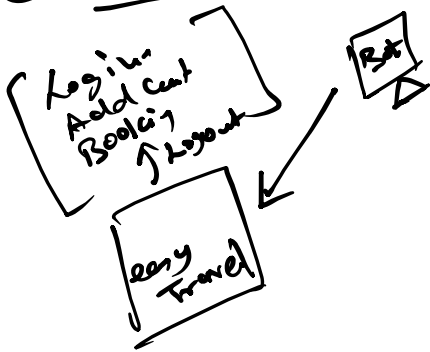


- ① Browser clickpath.
- ② Adv. feature of Synthetic Monitoring
- ③ SLO (Service level objective)
- ④ Log Analysis.

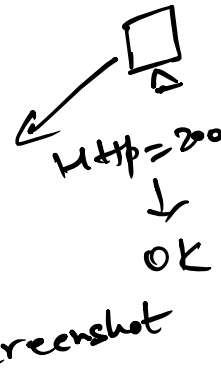
Dashboard & Report
Anomaly Detection.
Alert
Data's AT
Notebook

Last

① Browser clickpath :-



URL easy Travel



② Advance SLO :-

Get + full the value (fetch)
 Post → Add new resource. (create new data)
 Put → update the data.
 Patch → partial update
 Delete → Remove data. → Delete /users/101
 Head → get headers only-
 Option → show supported method
 ↳ get, post, put, patch, delete.
 Option /users

Get /users/101

Get / users / 101

Lifetch user 101

Post { user: { "name": " ", "email": " " } }

→ create new user

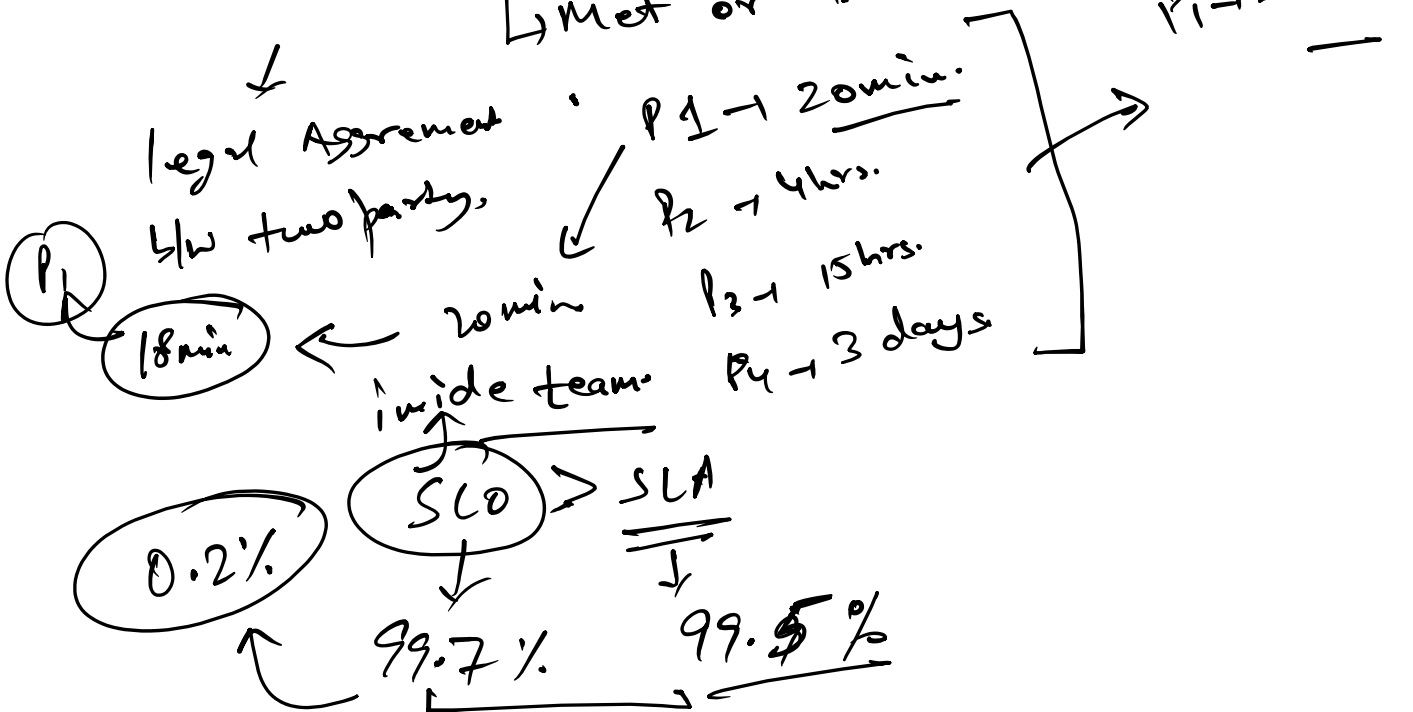
SLO - Service level objective.

SLI - Service level indicator (Memory = 40%, CPU = 75%)

SLO - Service level objective. (~~SLI~~ Inside team)

SLA - Service level Agreement

→ Met or Breach.



Error Budget:-

SLO = 99.7%

Error Budget = 0.3%

$$\text{Error budget} = \underline{\underline{(0.3\%)}}$$

Application can down
OR
upgrade OK TS

Type of SLO:-

- ① Availability SLO
- ② Latency (Performance) SLO
- ③ Failure SLO
- ④ Synthetic SLO
- ⑤ Step SLO

① Availability SLO:- Service is available & working successfully.

$$\text{Availability} = \left(\frac{\text{Successful Req.}}{\text{Total Req.}} \right)$$

EX:-

10,000 → Total req.

Failed req. → 50

Successful req = 9950

$$\text{Availability} = \left(\frac{9950}{10,000} \times 100 \right) = \underline{\underline{99.5\%}}$$

SA > 99.9%

④ Log Analysis:-

DBL

↳ Dynatrace Query Language.

① ~~data~~

② ~~Describe~~

③ ~~fetch~~

④ ~~timeseries~~

⑤ ~~fieldsAdd~~

⑥ ~~filterout~~

⑦ ~~filter~~

→ ⑧ alias → Rename.

⑨ ~~fieldRemove~~

⑩ to string

↓
Convert f1 → string

⑪ ~~append~~

⑫ ~~sort~~

⑬ ~~if-else~~

⑭ ~~fieldsRename~~

⑮ ~~contains~~

⑯ ~~endsWith~~

⑰ ~~Like~~ → %

↓
wildcard

① data:-

generate sample data during runtime.

Syn'r data[records], [json:'', son-string]

② Describe:-

On-read schema extraction def. for a given data object.

③ fetch →

Load data from specified data object.
ex:- fetch logs.

④ fieldsAdd → Add the field in data.

fieldsAdd {f1} {f2}:- - - -

⑤ filter out:- Exclude the item from the data.

⑤ filter out:- Exclude $\rightarrow$
!=
filter out (condition)

⑥ Summarize Aggregation
if-else countif
 $\downarrow$
Count the no. of event
on the specific criteria.

append:- output of Two diff. SQL query.

Sort = Sort - \$1.

if-else if (a > b) - Fields Add new field
{ print(a);
}
else { print(b);
}
= if (condition, then value,
else: else value)
 $\rightarrow$ = if (a > b, "a", else: "b")

contains Check whether the string has the sub-string.
... (1, 1, 1)

contains - Check whether the ...

contains(field, "Substr", CaseSensitive: false)

Tree Break

DPL :- Dyntaxe Pattern Lang.

Extract → Regular Expression

DPL

Where?

- ① Log Processing Rule in DT
- ② parse() command inside DQL queries
- ③ extracting field from ^{raw} log content at query origin.

STRING: Username → Match the string.

WORD: Service → Match single word

INT: StatusCode → Match an integer

FLOAT: responseTime → Match the float

IP: clientIP → Match IPv4 address.

LD: Match anything → LD: message.

EOL: End of line.

SPACE: Match whitespace.

... text.

✓
SPACE : Model with
'literal' : Matches exact text.